

Product Requirements Document (PRD)

Project Title

AI-Powered Resume Matcher & Optimizer with Version Tracking

1. Objective

Develop a web-based application that helps users optimize their resumes for a specific Job Description (JD) by:

- Uploading resumes (PDF/DOCX)
 - Uploading or pasting Job Descriptions
 - Calculating a Resume-JD Match Score
 - Identifying matched and missing skills
 - Generating AI-powered improvement suggestions
 - Tracking multiple resume versions and their scores
 - Allowing users to view previous resumes and compare improvements
-

2. Problem Statement

Students and job seekers often submit generic resumes that are not tailored to a specific job description.

This results in:

- Lower ATS scores
- Reduced interview opportunities
- Lack of visibility into missing skills
- Difficulty measuring resume improvements over time

The proposed system provides automated analysis and feedback to help users improve their resumes before applying.

3. Target Users

Primary Users

- Students
- Fresh graduates

- Job seekers
-

4. Core Features

Feature 1: Resume Upload

Users can upload:

- PDF files
- DOCX files

System will:

- Validate file type
 - Store original resume file
 - Extract resume text
 - Create a new resume version
-

Feature 2: Job Description Upload

Users can:

- Paste Job Description text
OR
- Upload Job Description document

System will:

- Extract keywords
 - Extract required skills
 - Extract role information
-

Feature 3: Resume Parsing

System extracts:

- Skills
- Education
- Projects
- Certifications

- Experience
-

Feature 4: Resume Matching

System compares Resume and Job Description using NLP techniques.

Matching Criteria:

- Skill Match
- Keyword Match
- Experience Relevance
- Project Relevance

Output:

- Overall Match Score (%)
-

Feature 5: Missing Skill Detection

System identifies:

Matched Skills

Example:

- Java
- Spring Boot
- SQL

Missing Skills

Example:

- Docker
 - Kubernetes
 - AWS
-

Feature 6: AI Suggestions

System generates recommendations such as:

- Add missing technical skills
- Improve project descriptions

- Use stronger action verbs
 - Add measurable achievements
 - Improve ATS compatibility
-

Feature 7: Resume Version Tracking

Every uploaded resume is stored as a separate version.

Example:

Version 1 → 62%

Version 2 → 78%

Version 3 → 91%

For each version:

- Resume file is preserved
- Upload date is stored
- Match score is stored
- Suggestions are stored

Users can:

- View previous resumes
 - Download previous resumes
 - Compare scores across versions
-

Feature 8: Dashboard

Display:

Resume Analysis

- Match Score
- Matched Skills
- Missing Skills
- AI Suggestions

Version History

- Resume Version
- Upload Date

- Score
-

5. Technology Stack

Frontend

- React
- Axios
- React Router

Backend

- FastAPI
- Python

Database

- PostgreSQL

ORM

- SQLAlchemy

AI/NLP

- Sentence Transformers
- spaCy

File Processing

- PyMuPDF
- python-docx

Deployment

- Frontend: Vercel
 - Backend: Render
 - Database: Neon PostgreSQL
-

6. Database Design

users

- id
 - name
 - email
 - password
 - created_at
-

job_descriptions

- id
 - title
 - description
 - created_at
-

resume_versions

- id
 - user_id
 - jd_id
 - version_no
 - file_name
 - file_path
 - extracted_text
 - uploaded_at
-

match_results

- id
 - resume_version_id
 - score
 - matched_skills
 - missing_skills
 - ai_suggestions
 - created_at
-

7. File Storage Strategy

Actual resume files will not be stored in PostgreSQL.

Resume files will be stored in server storage.

Example:

uploads/

user_101/

resume_v1.pdf

resume_v2.pdf

resume_v3.pdf

PostgreSQL stores:

- File name
- File path
- Version number
- Score
- Metadata

8. API Endpoints

Resume

POST /upload/resume

GET /resume/{id}

GET /resume/history/{userId}

Job Description

POST /upload/job-description

Matching

POST /match

GET /results/{resumeVersionId}

System

GET /health

9. User Workflow

1. User uploads Resume
 2. User uploads or pastes Job Description
 3. System extracts text
 4. System analyzes Resume and JD
 5. System calculates Match Score
 6. System identifies matched and missing skills
 7. System generates AI suggestions
 8. Results are displayed
 9. Resume version is stored
 10. User uploads improved resume
 11. New version score is generated
 12. Improvement history is displayed
-

10. Success Criteria

The project will be considered successful if:

- Resume text extraction works correctly
- Job Description parsing works correctly
- Match score is generated successfully
- Missing skills are identified accurately
- AI suggestions are generated
- Multiple resume versions are stored
- Previous resumes can be viewed/downloaded
- Score improvements can be tracked over time
- Complete workflow functions end-to-end